

Week 9 Problem Set

String Algorithms, Approximation

1. (Boyer-Moore algorithm)

a. Implement a C-function

```
int *lastOccurrence(char *pattern, char *alphabet) { ... }
```

that computes the last-occurrence function for the Boyer-Moore algorithm. The function should return a newly created dynamic array indexed by the numeric codes of the characters in the given alphabet (a non-empty string of ASCII-characters).

Ensure that your function runs in $O(m+s)$ time, where m is the size of the pattern and s the size of the alphabet.

Hint: You can obtain the numeric code of a `char c` through type conversion: `(int)c`.

b. Use your answer to Exercise a. to write a C-program that:

- prompts the user to input
 - an alphabet (a string),
 - a text (a string),
 - a pattern (a string);
- computes and outputs the last-occurrence function for the pattern and alphabet;
- uses the Boyer-Moore algorithm to match the pattern against the text.

An example of the program executing could be

```
prompt$ ./boyer-moore
Enter alphabet: abcd
Enter text: abacaabadcabacabaabb
Enter pattern: abacab

L[a] = 4
L[b] = 5
L[c] = 3
L[d] = -1

Match found at position 10.
```

If no match is found the output should be: `No match.`

Hints:

- You may assume that
 - the pattern and the alphabet have no more than 127 characters;
 - the text has no more than 1023 characters.
- To scan `stdin` for a string with whitespace, such as "a pattern matching algorithm", you can use:

```
#define MAX_TEXT_LENGTH 1024
#define TEXT_FORMAT_STRING "%[^\n]*c"

char T[MAX_TEXT_LENGTH];
scanf(TEXT_FORMAT_STRING, T);
```

This will read every character as long as it is not a newline `'\n'`, and `"%*c"` ensures that the newline is read but discarded.

We have created a script that can automatically test your program. To run this test you can execute the `dryrun` program that corresponds to this exercise. It expects to find a program named `boyer-moore.c` in the current directory. You can use `autotest` as follows:

```
prompt$ 9024 dryrun boyer-moore
```

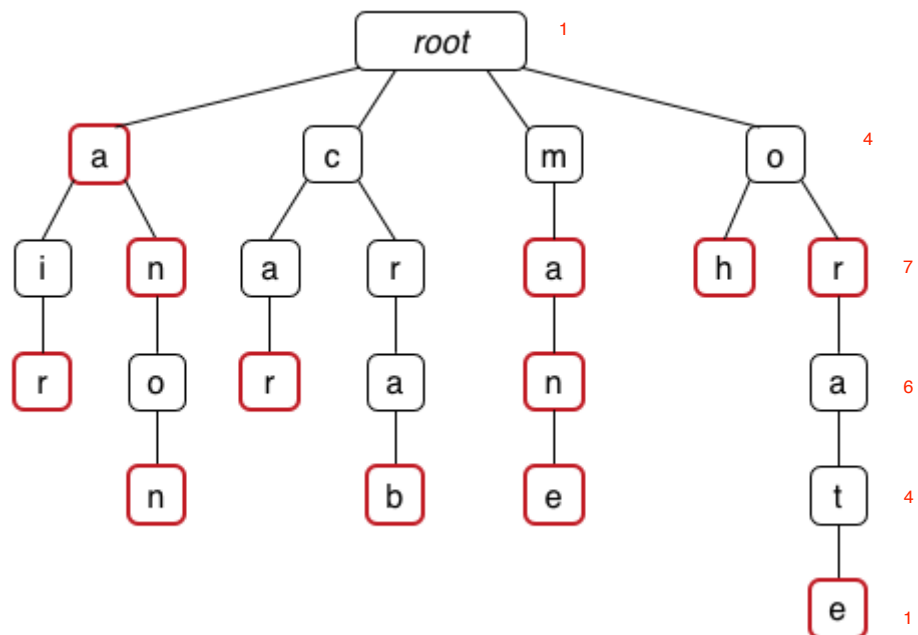
2. (Knuth-Morris-Pratt algorithm)

Develop, in pseudocode, a modified KMP algorithm that finds every occurrence of a pattern P in a text T . The algorithm should return a queue with the starting index of every substring of T equal to P .

Note that your algorithm should still run in $O(n+m)$ time, and it should find every match, including those that "overlap".

3. (Tries)

a. Consider the following trie, where finishing nodes are shown in red:



What words are encoded in this trie?

b. If the following keys were inserted into an initially empty trie:

```

jaws boots axe boo so jaw sore boot boon
  
```

what would the final trie look like? Does the order of insertion matter?

c. Answer question b. for a compressed trie.

4. (Text compression)

Compute the frequency array and draw a Huffman tree for the following string:

```

dogs do not spot hot pots or cats
  
```

5. (Numerical approximation)

Write a C program to implement the [root finding approximation algorithm](#) from the lecture as the function:

```
double bisection(double (*f)(double), double x1, double x2)
```

Use your program to find roots for

a. $f(x) = x^3 - 7x - 6$, in the interval $[0.0, 10.0]$

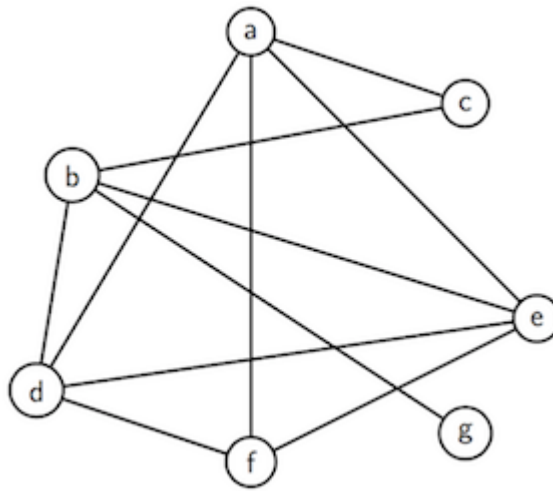
b. $f(x) = \sin x$, in the interval $[2.0, 4.0]$

c. $f(x) = \sin 5x + \cos 10x + \frac{x^2}{10}$, in the intervals $[0.0, 1.0]$ and $[1.0, 2.0]$

with precision $\epsilon=10^{-10}$.

6. (Vertex cover)

a. Apply the [approximation algorithm for vertex cover](#) to the following graph:



Find two different executions, one that results in an optimal cover and one that does not.

b. What size is an optimal vertex cover for complete graphs K_n ? Does the approximation algorithm always find an optimal cover for complete graphs?

c. A *complete bipartite graph* $K_{m,n}$ is an undirected graph that has:

- two disjoint vertex sets V_m and V_n of size $m \geq 1$ and $n \geq 1$, respectively;
- every possible edge connecting a vertex in V_m to a vertex in V_n ;
- no edge that has both endpoints in V_m or both endpoints in V_n .

Answer question b. for complete bipartite graphs.

7. (Feedback)

We (Michael and Michael, David and Sahil) want to hear from you how you liked COMP9024 and if you have any suggestions on how the course should be run in the future.

Log in to [MyExperience](#) to provide feedback. Please do so even if you just want to say, "I liked the way it was taught".

8. Challenge Exercise

Given a string s with repeated characters, design an efficient algorithm for rearranging the characters in s so that no two adjacent characters are identical, or determine that no such permutation exists. Analyse the time complexity of your algorithm.

Assessment

After you've solved the exercises, go to [COMP9024 19T3 Quiz Week 9](#) to answer 5 quiz questions on this week's problem set (Exercises 1-7 only) and lecture.

The quiz is worth 2 marks.

The deadline for submitting your quiz answers is **Monday, 18 November 11:00:00am**.

As always please be mindful of the **quiz rules**:

Do ...

- use your own best judgement to understand & solve a question
- discuss quizzes on the forum only **after** the deadline on Monday

Do not ...

- post specific questions about the quiz **before** the Monday deadline
- agonise too much about a question that you find too difficult